

Capability-Oriented Software Engineering (COSE): An Incremental Methodology for Building Reusable Capability Libraries (Version 3.0)

Enrique Solis Carvajal

Change Tensors Solutions — Systems Architecture & Engineering

Email: esolis@tcgplanet.cl

This research was conducted within the COSE Multi-Agent and Systems Architecture Infrastructure.

Version history: v1.0 (June 2026), v2.1 (August 2026), v3.0 (September 2026). Canonical repository: github.com/Solis666/cose-methodology. DOI: 10.5281/zenodo.22029421.

Abstract—Despite decades of progress in software engineering paradigms, software development projects frequently suffer from architectural re-invention, encapsulating functional knowledge within individual project silos. This paper presents Capability-Oriented Software Engineering (COSE), a formal incremental methodology designed to systematically convert software engineering effort into a cumulative, permanent library of reusable, decoupled capabilities. COSE establishes a mathematical framework for problem decomposition into single-responsibility micro-problems, resolved via deterministic or cognitive capabilities assembled into execution pipelines. Under COSE, large language models (LLMs) are treated strictly as interchangeable, bounded cognitive components within deterministic control structures, preserving model independence and auditability via persistent file-based artifacts. We evaluate COSE across a diverse spectrum of real-world production systems: Mammon (an algorithmic trading platform validated across a 36-configuration stress matrix and live-paper trading on ETHUSD), Knowledge Factory (a hybrid cognitive distillation pipeline using structured JSON tensors), Website Intelligence Factory (WIF, web reverse-engineering and automated crawler generation), a multi-model AI Middleware orchestrator, and commercial business applications (Dol-Goldur, CatalogVision, and Andigraf). Version 3.0 extends the empirical basis in four directions. First, it formalizes a *physical evidence-based reconstruction methodology*, derived from a six-phase audit of a live engineering ecosystem, establishing that declared project state must be reconstructed from execution artifacts rather than from documentation. Second, it introduces a capability messaging bus that replaces monolithic orchestration with a minimal-routing courier over decoupled capabilities. Third, it adds a sixth architectural principle addressing task-state persistence across non-persistent model invocations, together with a clarification that agent granularity and capability granularity are orthogonal design dimensions. Fourth, it adds an eighth case study documenting autonomous objective discovery under open-ended monetization directives, including a reflexive cross-verification episode in which the system detected and corrected a misattribution of its own output. The empirical evidence demonstrates complete infrastructure decoupling, robust operational stability under variable execution parameters, and a progressive reduction in the marginal cost of building complex software systems through composition rather than reconstruction.

Index Terms—Software Engineering Methodology, Reusable Capabilities, Multi-Agent Systems, Pipeline Composition,

Deterministic-Cognitive Decoupling, Large Language Models, Evidence-Based Reconstruction, Capability Messaging, Task-State Persistence, Autonomous Objective Discovery.

I. INTRODUCTION

A. Origin of the Methodology

Software engineering has experienced continuous evolution over the past decades. Paradigms such as structured programming, object-oriented design, service-oriented architecture (SOA), microservices, and multi-agent systems have progressively aimed to manage the complexity of building sophisticated computing solutions. However, a recurring inefficiency persists: each new software project frequently restarts substantial portions of functional design, even when similar problems have already been solved in previous contexts.

In traditional development workflows, knowledge acquired during implementation remains encapsulated within project-specific codebases, application-bound components, and tacit developer experience. Consequently, organizations lack a systematic mechanism to lower the marginal cost of future software construction.

The methodology presented in this paper did not originate from an abstract theoretical exercise in multi-agent systems or software architecture. It emerged inductively from industrial practice. The author, trained as a business administrator without formal education in computer science or software engineering, began experimenting with large language models (LLMs) and software construction approximately ten months prior to this work. An initial system, informally named *Los Cabros*, was designed around an organizational analogy: interchangeable “characters” (inspired by narrative archetypes), each possessing distinct skills, to be composed dynamically according to the requirements of a given task. This early multi-agent intuition reflected a business-oriented understanding of specialization and collaboration—an approach later documented in the ecosystem context that serves as the workspace within which the COSE projects operate. In practice, however, the character-based approach proved over-qualified for many of the concrete problems being addressed; individual agents frequently concentrated capabilities that could be executed more simply.

A decisive shift occurred during the development of Dol-Goldur, an automation system for catalog enrichment and

website publication for a physical music retail operation. Instead of assigning broad skill sets to agents, the workflow was decomposed into fine-grained, single-responsibility micro-capabilities arranged in an explicit pipeline. Subsequent systems—CatalogVision (image-to-inventory processing) and Andigraf (competitive price intelligence and reporting)—confirmed the same pattern: micro-capabilities proved easier to audit, debug, and, crucially, reuse across projects, in contrast to the more complex and less transferable character constructs. The repeated observation that functional units developed for one domain could be composed into solutions for another led to the recognition that a systematic methodology was emerging.

This methodology was then formalized as Capability-Oriented Software Engineering (COSE) and subsequently applied to larger-scale systems, including Website Intelligence Factory (web reverse-engineering and crawler generation), Knowledge Factory (hybrid cognitive distillation), Mammon (algorithmic trading infrastructure), and a multi-model AI Middleware orchestrator.

B. Core Tenets

COSE rests on four core tenets that crystallized from this trajectory:

- 1) **Systematic Problem Decomposition:** Complex domain problems are recursively decomposed into discrete, single-responsibility micro-problems.
- 2) **Atomic Capabilities:** Micro-problems are resolved by minimal, autonomous functional units termed capabilities, categorized strictly as either deterministic (invariant code, rules, transformations) or cognitive (LLM-driven reasoning, judgment, or language synthesis).
- 3) **Cumulative Capability Library:** Capabilities do not belong to specific projects; they are permanently registered into a central, evolving Capability Library.
- 4) **Dynamic Pipeline Composition:** Complex software applications are assembled dynamically by chaining existing capabilities into execution pipelines governed by persistent disk-based artifacts.

In COSE, the fundamental unit of organizational value shifts from the transient application deliverable to the permanent, cumulative Capability Library. The methodology maintains strict technology and model independence: LLMs serve as replaceable cognitive engines within deterministic control structures, preventing architectural lock-in and ensuring deterministic execution bounds.

C. Physical Evidence-Based Reconstruction Methodology

Version 3.0 formalizes a reconstruction procedure that emerged from the audit of a live engineering ecosystem comprising the systems evaluated in Section V. The procedure addresses a systematic discrepancy observed between project documentation and the effective state of the artifacts it describes.

During the audit, project logs consistently declared states that physical inspection of the workspace did not support. Two

classes of discrepancy were identified. In the first class, work had advanced *beyond* what the log recorded: a component documented as awaiting validation had in fact completed a full end-to-end execution whose artifacts remained in the runtime directories. In the second class, work was *ongoing* without appearing in the log at all: a continuous production validation running in the operational environment was absent from the documentation because it had been started outside the logging workflow.

Both classes share a single root cause: documentation was being treated as the authoritative record of system state, when it is in fact a historical and inherently incomplete record of intent. The methodology therefore adopts the following principle:

The log is the plan; the filesystem is reality.

The corresponding operational rule is normative: *no state of a component may be declared complete, pending, or failed on the basis of documentation alone*. The effective state of a component must be reconstructed by contrasting the documentation against physical evidence — execution artifacts in runtime directories, structured output artifacts, logs carrying timestamps, code structure, and declared capability contracts. Documentation is retained as historical reference and as a statement of intent; physical evidence is the operational source of truth.

A representative reconstruction illustrates the procedure. A web reverse-engineering system appeared in its project log, whose most recent entry predated the audit by several weeks, as having its physical validation pending. Inspection of the system’s runtime directory contradicted this state: the pipeline had executed end-to-end in a single run, producing thirteen categories of raw evidence, nine independent inspection detectors, a site profile, an extraction profile, a generation plan, a fully materialized Python crawler project, and a validation report reporting `passed: true` across structural, strategic, contractual, and traceability checks. The reconstruction therefore reclassified the component from *validation pending* to *validation complete, crawler generated and validated*. The same inspection also surfaced a qualification that documentation alone would have obscured: the effective target was a challenge endpoint protected by a web application firewall (HTTP 403, evasion strategy `stealth_browser` recommended by the system’s own planning capability). The chain worked end-to-end, but on a degenerate target, and execution of the *generated* crawler against the live site remained unverified. Reconstruction thus produced not merely a corrected status but a more accurate boundary of what had and had not been demonstrated.

A second finding of the audit is directly relevant to the cost argument of the methodology. Reconstruction effort proved substantially lower in components already structured under COSE than in monolithic components of comparable size. The reason is structural rather than incidental: capability contracts and manifests declare inputs, outputs, and permissions as artifacts rather than as prose; execution results persist as verifiable structured files; and governed capability mailboxes

leave a physical trace of every invocation. In such components, determining state is a matter of locating and verifying artifacts. In monolithic components, where no such artifacts exist, the same determination requires reconciling source code against narrative documentation against dispersed execution evidence. The audit concluded that COSE materializes traceability as an architectural property, and that the cost of reconstructing system state is therefore an architectural outcome rather than a documentation outcome.

The procedure is presented here as a methodological contribution rather than as project hygiene, because it defines the validation standard applied throughout Section V: an empirical claim in this paper is admitted only when it is supported by a physical artifact, and the qualification of that artifact’s scope is reported together with the claim.

D. Contributions of Version 3.0

Relative to the published Version 2.1, this revision contributes the following:

- 1) **Physical Evidence-Based Reconstruction Methodology** (Section I-C), with the normative rule that effective state is reconstructed from physical evidence rather than from documentation.
- 2) **Empirical update of the case studies** (Section V) based on the reconstruction audit: a completed and validated pipeline execution for the web reverse-engineering system, an active continuous live-paper validation for the trading platform, and the operational state of the multi-model middleware.
- 3) **The COSE Conformity Matrix** (Section V-G), a six-component $\times$ six-principle instrument used as an internal audit standard, with per-cell justification.
- 4) **The COSE capability messaging bus** (Section IV), a pure-courier architecture over decoupled capabilities, with an address book, per-capability inboxes and outboxes, and capability-level governance.
- 5) **An explicit distinction between canonical principles and local implementation conventions**, so that adoption quality is evaluated against the principles of the methodology rather than against any particular implementation’s conventions.
- 6) **A copy-and-deprecate migration strategy** (Appendix B), in which the running infrastructure is not modified during decoupling: work proceeds on a copy that becomes the implemented artifact while the original is retained as deprecated evidence of the prior state.
- 7) **The Canonical Backlog as a governance capability** (Appendix C), a prospective task-state artifact distinct from both the execution log and the messaging bus.
- 8) **Clarification of agent-capability orthogonality** (Section III-B), and an additional architectural principle addressing task-state persistence across non-persistent model invocations (Section IV).

The remainder of this paper is organized as follows. Section II reviews related work. Section III formalizes the theoretical framework, mathematical notation, and glossary, including the

formal definition of the task-state artifact and the clarification of agent and capability granularity. Section IV details the COSE architectural principles, the capability messaging bus, and the execution lifecycle. Section V presents empirical evaluations across seven production systems and one open-ended autonomy case study, together with the conformity matrix. Section VI offers a comparative analysis, a discussion of findings, and the limitations of the methodology. Section VII concludes with key insights and future research directions. Appendices A, B, and C document open-science traceability, the infrastructure decoupling route, and the execution governance model, respectively.

II. RELATED WORK

The paradigm of Capability-Oriented Software Engineering (COSE) intersects three major domains of software engineering and artificial intelligence: multi-agent systems, tool-augmented language models, and architectural patterns for reusability.

A. Multi-Agent Systems and Agentic Frameworks

Distributed Artificial Intelligence and Multi-Agent Systems (MAS) established the foundations for autonomous agent cooperation and task allocation [1]. With the advent of Large Language Models (LLMs), agentic frameworks such as ReAct [2] and AutoGen [3] demonstrated effective reasoning, planning, and multi-agent communication. However, these frameworks primarily focus on dynamic task execution within a single session or context window. They lack explicit mechanisms to transform execution learnings into permanent, cross-project functional libraries, leading to recurring engineering overhead across projects.

B. Tool-Augmented LLMs and Function Calling

Augmenting LLMs with external tools enables models to execute actions beyond text synthesis. Toolformer [4] demonstrated self-supervised learning for tool usage, while ToolLLM [5] scaled tool interaction to thousands of real-world APIs. Despite these advances, tool-use approaches treat external tools as execution extensions of the LLM. In contrast, COSE treats the LLM strictly as a bounded cognitive component embedded within a deterministic control structure, enforcing that deterministic logic remains completely independent of model behavior.

C. Software Engineering Paradigms and Neurosymbolic Integration

Component reusability and functional decoupling are classic principles in software architecture. Microservices [6] promote service-level independence, while the Unix philosophy [7] advocates minimal, composable programs. In the context of AI-driven systems, Kautz [8] emphasized the necessity of integrating neural learning with symbolic reasoning—a principle known as the “third wave” of AI. COSE operationalizes this integration by formalizing a strict separation between deterministic capabilities (symbolic/invariant) and cognitive

capabilities (neural/LLM), mediated via persistent, file-based artifacts.

D. Task-State Continuity and Reflexive Cross-Verification

Two properties distinguish COSE from the paradigms reviewed above, and both were identified empirically rather than assumed: the treatment of unresolved task state across non-persistent model invocations, and the application of cross-verification to the system’s own claims about its output.

Agentic frameworks are designed around execution within a bounded session or context window [2], [3]. A consequence observed in sustained operation of our own infrastructure is that an unresolved reasoning chain or a pending decision generated within one model invocation does not, by default, remain bound to the responsibility of resolving it in a subsequent invocation. The obligation lapses together with the context that produced it, rather than being carried forward as it would be for an agent with persistent memory. The practical effects are loss of traceability, duplicated exploratory work, and the silent expiration of items awaiting an external event. COSE addresses this in Section IV through a persistent, external task-state artifact, whose function is exclusively prospective and which is formally distinct from both the execution log and the capability messaging bus.

The second property concerns verification direction. COSE’s artifact-based design makes claims about system state independently checkable, which permits the same verification discipline to be applied reflexively—to the system’s own assertions about what it produced, rather than only to its declared project state. Section V-F documents a concrete instance: a system instance misattributed a pre-existing third-party contribution as its own output during an audit pass, and a second instance, using programmatic platform metadata (authorship and timestamp), detected and corrected the error before it propagated into a final report. This episode is treated in Section VI as evidence that the evidence-over-documentation principle extends to statements that an autonomous system makes about itself.

III. THEORETICAL FRAMEWORK & NOTATION

To eliminate ambiguity and establish formal rigor, this section defines the mathematical notation, mathematical relations, and core domain terminology of the COSE methodology.

A. Formal Definitions

- **Problem (P):** A concrete functional requirement whose resolution produces operational value, defined independently of implementation technology.
- **Micro-problem (M):** The minimal, single-responsibility functional unit derived from the decomposition of a problem.
- **Capability (C):** An autonomous, reusable, and deterministic or cognitive functional unit designed to solve a specific micro-problem.
- **Capability Library (L):** The cumulative, non-decreasing set of all validated capabilities available to the organization.

- **Agent (A):** An executable mechanism or runtime entity — deterministic or cognitive — that invokes one or more capabilities in service of resolving a problem P or a subset of its decomposed micro-problems. An agent is not, by definition, a cognitive (LLM-driven) entity: a fully deterministic pipeline orchestrating an arbitrary number of deterministic capabilities constitutes an agent in the same formal sense as a cognitive model issuing tool calls. The number of capabilities an agent invokes is determined exclusively by the requirements of the task at hand, not by an a priori architectural constraint. Website Intelligence Factory (Section V-C) exemplifies this: its orchestrating pipeline is itself a deterministic agent coordinating over thirty independent deterministic capabilities.
- **Pipeline (II):** An ordered sequence of capabilities executing data transformations, where output artifacts from step i serve as input artifacts for step $i + 1$.
- **Artifact (T):** A persistent, file-based data structure (e.g., JSON schema, CSV tensor, report, image) generated during pipeline execution that guarantees state auditability.
- **Task-State Artifact (S):** A persistent, external structure recording the obligations that remain unresolved across execution boundaries. It is prospective rather than historical — it records what must still occur and why, not what has already occurred — and it is formally distinct from both the execution log, which is a passive historical record, and the messaging bus, which routes work between capabilities. Its entries, state semantics, and persistence property are formalized in Section III-B.
- **Orchestrator (O):** The component responsible for routing data flows, managing state transitions, and sequencing capability execution without embedding domain business logic.

B. Mathematical Notation and Algebra

Let $P \in \mathcal{P}$ represent a target domain problem. The decomposition operator $decompose(\cdot)$ maps P to an ordered set of micro-problems:

$$decompose(P) \longrightarrow \{M_1, M_2, \dots, M_n\} \quad (1)$$

For each micro-problem M_i , the resolution function $resolve(M_i)$ queries the current Capability Library L :

$$resolve(M_i) = \begin{cases} C_k \in L, & \text{if a matching capability exists} \\ C_{new}, & \text{otherwise} \end{cases} \quad (2)$$

When a matching capability does not exist in L , a new capability C_{new} is developed and permanently ingested into L via the library expansion operator:

$$L_{t+1} = L_t \cup \{C_{new}\} \quad (3)$$

Thus, the cardinality of the Capability Library grows monotonically over time:

$$|L_{t+1}| \geq |L_t| \quad (4)$$

A complete software application or workflow is constructed by composing capabilities into an executable pipeline Π :

$$\Pi = \text{compose}(C_1, C_2, \dots, C_n) \quad (5)$$

where data flow between capability nodes is mediated exclusively through persistent artifacts T :

$$\Pi : T_{in} \xrightarrow{C_1} T_1 \xrightarrow{C_2} T_2 \dots \rightarrow T_{n-1} \xrightarrow{C_n} T_{out} \quad (6)$$

An agent A_j executes a set of capabilities:

$$A_j \longrightarrow \{C_a, C_b, \dots\} \quad (7)$$

Agent granularity and capability granularity are orthogonal design dimensions. An agent may orchestrate an arbitrary number of capabilities within a single execution context; the Single Responsibility constraint (Section IV-A.1) is enforced strictly at the capability level, not at the agent level. A cognitive agent — particularly one embodied by an LLM issuing tool calls — is expected to invoke multiple atomic capabilities in the course of resolving a single problem P , provided each invoked capability independently satisfies atomic, single-responsibility scope. Conversely, a deterministic agent may orchestrate a large library of deterministic capabilities with no cognitive component involved in its control flow; the only constraint on the number of capabilities an agent invokes is the structure of the problem being resolved.

Model independence is mathematically defined by stating that replacing the underlying LLM provider ($LLM_1 \rightarrow LLM_2$) leaves the methodology and deterministic capability interfaces invariant:

$$\text{replace}(LLM_1, LLM_2) \implies \text{Methodology and Pipeline Invariant} \quad (8)$$

The task-state artifact S is defined as a persistent set of task-state items:

$$S = \{s_1, s_2, \dots, s_m\} \quad (9)$$

Each item carries three components, and is written as the triple:

$$s = (\sigma(s), \rho(s), \varepsilon(s)) \quad (10)$$

where $\sigma(s) \in \Sigma$ is the discrete state of the item, drawn from a finite state set

$$\Sigma = \{\text{active, pending-human-action, standby, under-review, frozen, completed, discarded}\}$$

$\rho(s)$ is the re-emergence condition of the item — an event predicate or a deadline — such that a standby item which satisfies $\rho(s)$ is escalated for review rather than silently expiring; and $\varepsilon(s)$ is a reference to verifiable evidence supporting the item's current state.

Two properties distinguish S from the artifact family T and from the execution log. The first is *obligation persistence*: if an invocation k terminates with an item unresolved, the item is not discarded but carried forward in S , so that

$$\sigma(s) \notin \{\text{completed, discarded}\} \text{ at close of } k \implies s \in S_{k+1} \quad (11)$$

The second is *active re-emergence*: an item whose re-emergence condition is satisfied is delivered as an incoming request through the capability messaging mechanism (Section IV-B), rather than remaining passively available as background state, so that

$$\rho(s) \text{ holds at invocation } k' \implies s \text{ is delivered as an incoming request at } k' \quad (12)$$

Equation (11) addresses the loss of unresolved work across non-persistent invocations; Equation (12) addresses the observed difference between an obligation that is merely recorded and one that is actively re-presented. Both properties are stated here as the formal content of the architectural principle developed in Section IV-A.6.

IV. ARCHITECTURAL PARADIGM

The COSE architectural framework enforces strict segregation of concerns across system design, runtime execution, and persistence layers.

A. Core Design Principles

- 1) **Single Responsibility and Atomic Granularity:** Each capability C_i solves exactly one micro-problem M_i . Complex behavior must emerge from composition, not from internal capability bloating.
- 2) **Strict Deterministic vs. Cognitive Segregation:**
 - **Deterministic Capabilities:** Executed via immutable code, mathematical algorithms, database queries, or logical rules. Inputs and outputs are predictable and invariant given identical state.
 - **Cognitive Capabilities:** Executed by LLMs for tasks requiring non-deterministic reasoning, ambiguity resolution, or natural language generation.
 - **Segregation Rule:** Any computation that can be executed deterministically must be removed from cognitive units.
- 3) **Artifact-Based State Persistence:** Inter-capability communication occurs strictly through persistent file artifacts stored in the workspace repository. In-memory volatile contexts across execution boundaries are prohibited, ensuring full execution replayability and auditability.
- 4) **LLM as a Bounded Component:** LLMs are treated strictly as isolated cognitive workers invoked at specific pipeline nodes, rather than acting as unconstrained, autonomous system controllers.
- 5) **Model Independence and Interchangeability:** Cognitive capabilities interface through standardized client abstractions, enabling runtime swapping between local

open-weight models (e.g., Ollama/DeepSeek) and commercial cloud APIs (e.g., OpenAI/Gemini) without modifying surrounding capabilities or orchestration logic.

- 6) **Task-State Persistence Across Non-Persistent Model Invocations:** Standard LLM-based agent invocations lack persistent memory across independent context windows. Consequently, an unresolved reasoning chain or pending decision generated within one invocation does not, by default, remain bound to the responsibility of resolving it in a subsequent invocation: the obligation evaporates along with the context that produced it, rather than being carried forward as it would be for an agent with persistent memory.

COSE addresses this via a persistent, external task-state artifact (Section III-A), structurally distinct from both the execution log (a passive historical record) and the capability-messaging bus (Section IV-B, responsible for inter-capability routing). Its function is exclusively prospective: it records what must occur and why, not what has already occurred. Each entry carries (a) a discrete state, (b) a re-emergence condition or timeout after which a standby item is escalated for review rather than silently expiring, and (c) a reference to verifiable evidence supporting its current state.

This mechanism serves two complementary functions. First, it reduces contextual load by allowing an unresolved item to be removed from active reasoning without being lost. Second, and more significantly, it re-binds the obligation to resolve an unresolved reasoning chain to the same responsibility locus that generated it, rather than allowing that obligation to lapse when the originating context window closes — substituting an artifact-based analogue for the accountability continuity that persistent memory would otherwise provide.

B. The Capability Messaging Bus

Principles 1 and 2 constrain how work is decomposed; the question of how decomposed capabilities receive work is an architectural question of its own. Version 2.1 documented orchestration through a single controller that invoked capabilities directly. The architecture described here replaces that arrangement with a messaging bus whose components are themselves capabilities and artifacts, and whose defining property is that the courier knows nothing about the logic of what it transports.

Three properties follow from this design, and all three were validated in operation (Section V-D). The first is *declarative extension*: adding a capability consists of declaring it in the address book, creating its mailbox, and providing its executor — the courier is not modified, and no existing capability is rebuilt. The second is *routing transparency*: capabilities contain no hard-coded paths, because destinations are resolved by the courier against the address book rather than by the capability against the file system. The third is *governed execution*: each capability carries a manifest declaring its identity and contracts, and a governance file declaring what it may

```
[Cognitive Models / Clients] --deposit
message--> bus/inbox/*.json
|
v
COURIER (pure) <--address book
(declarative)-->
- validates structure (id, origin,
destination, type, payload)
- resolves destination against the address
book
- copies the message to
bus/inboxes/<capability>/
- archives the original; appends to the
routing log
|
v
CAPABILITY (own daemon) observes
bus/inboxes/<capability>/
- deterministic execution under its own
governance contract
- result returned as a structured artifact
in bus/outboxes/<capability>/
```

Fig. 1. The COSE capability messaging bus: a pure courier resolving destinations through a declarative address book, with per-capability inboxes and outboxes.

read, where it may write, which operations it may perform, and what it must refuse. Refusals are first-class results — a rejected operation returns a structured negative outcome and produces an audit record — so that the boundary of authority is itself a persisted artifact.

The bus is deliberately minimal, and this minimality is the architectural claim: the transport layer holds no domain logic, no scheduling policy, and no task semantics, which is what allows capabilities to be added, governed, and independently audited without touching it.

C. Prospective Task-State Governance

Principle 6 requires a task-state artifact, and the bus provides the delivery medium for it, but neither specifies who maintains the state. In the architecture described here, that responsibility belongs to a dedicated governance capability whose central artifact records the tasks the system owes, their state, their re-emergence conditions, and the evidence supporting each state.

Two properties of the governance capability are relevant to the methodology. First, it is *separate from the messaging bus*, and the separation is not incidental: routing answers the question *how does work reach a capability*, whereas governance answers the question *what must still occur and why*. The two concerns evolved independently, and the separation of the bus from the ledger is itself an application of Principle 1 to the governance layer. Second, the capability embodies the states of Equation (10) as a strict single-state discipline — each task holds exactly one state at any time, state changes are transitions rather than deletions, and every transition must be justified by evidence, so that an abandoned or failed item becomes an explicitly discarded entry with its justification attached rather than disappearing. A health indicator completes the mechanism: an item whose re-emergence deadline passes

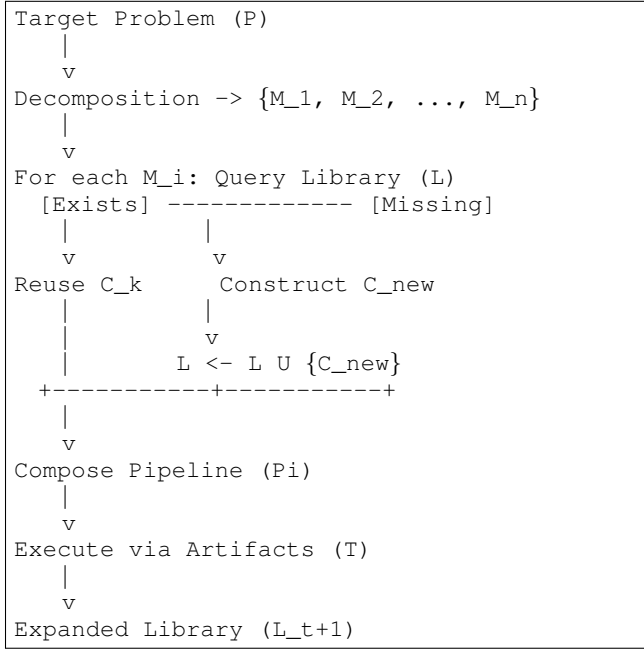


Fig. 2. The COSE System Lifecycle: Problem decomposition, capability retrieval/ingestion, pipeline composition, and artifact-based resolution.

is escalated to a review state, which means that unreviewed work surfaces as a signal instead of expiring in silence.

A preliminary observation from operating this mechanism is recorded here because it informed the design of the re-emergence rule, although its systematic quantification is left to future work: a task-state entry that is merely *read* within a subsequent context — available to the model as background information — appeared less effective at securing follow-through than the same entry *actively delivered* as an incoming request through the messaging bus once its re-emergence condition was met. This is the empirical motivation for Equation (12), where active delivery is a property of the mechanism rather than an operational habit. The governance model is documented in full in Appendix C.

D. The COSE Execution Lifecycle

The lifecycle is unchanged from Version 2.1 with respect to composition and artifact-mediated execution; what Version 3.0 adds is the obligation track that runs alongside it. At any point in the lifecycle, a micro-problem, a capability under construction, or a pipeline awaiting an external event may remain unresolved when the current invocation terminates. Under Principle 6, such items do not terminate with the invocation: they transition into the task-state artifact (Equation 11), remain governed by a re-emergence condition, and are returned to active reasoning through the messaging bus when that condition is met (Equation 12). The lifecycle therefore closes not on the expanded library alone, but on the expanded library together with the set of obligations the system has not yet discharged — an explicit, auditable account of what the system owes, which is the property that the artifact-based design makes available.

V. EMPIRICAL EVALUATION & CASE STUDIES

To validate the COSE methodology against real-world complexity, we evaluate its implementation across seven production systems developed within our engineering infrastructure, complemented in Section V-F by an eighth case study of open-ended autonomous operation.

A. Case Study 1 — Mammon: Algorithmic Trading Platform

1) *Problem Formulation*: Build an automated quantitative trading platform capable of data acquisition, candle validation, technical signal generation, paper execution, and historical backtesting, while isolating trading strategies as interchangeable components from underlying execution infrastructure.

2) *Decomposition & Pipeline Construction*: The system was decomposed into modular capabilities communicating via disk artifacts:

- `market.price_history`: Binance Spot REST API integration for raw OHLCV fetch.
- `market.candle_repository`: Deterministic candle validation, integrity check, and unified serving.
- `signal.entry`: Technical indicator strategy (RSI, MACD, Moving Averages) returning BUY/SELL/NEUTRAL.
- `execution.paper`: Simulated order matching, portfolio cash accounting, and order state persistence.
- `simulation.runner`: Historical backtesting engine over the complete pipeline.

Binance REST → price hist. → candle repo → signal.entry → exec.paper (13)

Mammon formally segregated operational data pipelines from knowledge-generation pipelines:

- **Operational Pipeline**: Fetch → Validate → Analyze → Execute.
- **Knowledge Pipeline**: Audit → Repository → Statistics → Interpretation → Hypotheses.

3) *Empirical Validation (36-Configuration Stress Matrix)*: To evaluate architectural stability, Mammon was subjected to a comprehensive stress matrix encompassing 36 distinct experimental configurations: Assets (4): BTCUSDT, ETHUSDT, SOLUSDT, BNBUSDT; Timeframes (3): 15m, 1h, 4h; Sample Windows (3): 200, 500, 1000 candles.

Results: The capability infrastructure achieved a 100% completion rate across all 36 configurations with zero infrastructure or state-desynchronization failures. Furthermore, during live paper execution on ETHUSDT (1h timeframe), a live data ingestion bug (frozen candle snapshot) was detected and repaired on August 10, 2026, by refactoring `simulation.runner` to pull fresh REST candles per cycle without modifying downstream signal or execution capabilities. The strategy’s selectivity principle—where returning NEUTRAL is treated as a valid decision—was empirically confirmed.

4) *Extended Validation — Continuous Live-Paper Operation (Version 3.0)*. Beyond the stress matrix, the operational

TABLE I
MAMMON INFRASTRUCTURE VALIDATION STRESS MATRIX (36 CONFIGURATIONS)

Asset	Timeframe	Window	Status	Failures
BTCUSDT	15m,1h,4h	200,500,1000	100% Pass	0
ETHUSDT	15m,1h,4h	200,500,1000	100% Pass	0
SOLUSDT	15m,1h,4h	200,500,1000	100% Pass	0
BNBUSDT	15m,1h,4h	200,500,1000	100% Pass	0

pipeline was subjected to an extended continuous live-paper validation on ETHUSDT (1h), reported here under the reconstruction procedure of Section I-C rather than from project documentation.

Establishing the boundaries of that validation required more care than the documentation implied, and the reason is itself an instance of the principle. The audited run was initialized on **2026-09-03 at 16:24:10 UTC** with its portfolio reset — Cash = \$10,000.00 | Equity = \$10,000.00 | PnL = \$0.00 | Position = NONE. That initialization record is taken from the operating console and is the authoritative boundary of the run, because the execution log cannot supply one: the log is a single file shared by successive runs and retains records written before the audited run began, so the log’s first entry is not the run’s first entry. Three properties of the file make the multi-run structure measurable, and each was obtained by reading the log directly rather than by inference:

- **A duplicated cycle block.** Immediately before the run’s first hourly evaluation, the log contains two consecutive blocks carrying the identical timestamp 2026-09-03T16:00:00+00:00. The duplication sits adjacent to a confirmed restart and is consistent with one — a process beginning at 16:24 could re-log the candle that had just closed — but its cause **cannot be attributed** from the available evidence: it may equally be a failed execution minutes earlier, or a log entry adjusted to the candle’s nominal time rather than the evaluation’s real time. It is reported as an unexplained duplication adjacent to a confirmed restart, not as evidence of one.
- **A change of record format.** The retained blocks are written in two mutually incompatible timestamp formats: 2026-08-28 20:47, without a zone, and 2026-08-30T19:00:00+00:00, in ISO form with an explicit offset. The change occurs at the block dated 2026-08-30T19:00, which demonstrates that the file was produced by more than one writer or writer version and therefore cannot be read as a single continuous record.
- **A cycle count that reproduces the reported totals.** The file contains **307 cycle blocks in total**, and their signal classification — **81 BUY / 55 SELL / 171 NEUTRAL** — matches the audited report’s figures exactly. The report’s declared filter begins on 2026-08-27, which precedes the file’s first entry, so the report aggregates everything the file contains rather than the audited run alone. The execution log therefore cannot be used to attribute the

TABLE II
MAMMON CONTINUOUS LIVE-PAPER VALIDATION — AUDITED STATE (2026-09-11, 19:00 UTC)

Metric	Reported Value
Pipeline	pipeline_eth_trailing_v1 (threshold 2.0 + ATR trailing 1.5×)
Symbol / Timeframe	ETHUSDT / 1h
Audited run	initialized 2026-09-03 16:24:10 UTC — portfolio reset (console record)
Audited-run window	2026-09-03T17:00 → 2026-09-11T19:00 UTC (≈8.1 days; 177 log blocks; ≈91% cadence; ±1 block)
Termination	host OS restart after security updates installed 2026-09-11
Preceding runs (same log)	130 blocks, 2026-08-28 20:47 → 2026-09-03T16:00; zero orders; cash flat at \$10,000.00
Reported totals (whole file; filter from 2026-08-27)	307 evaluations · 81 BUY / 55 SELL / 171 NEUTRAL
Orders executed	11 round trips (22 position transitions), entirely within the audited run
First / last order	2026-09-04 13:00 UTC / 2026-09-11 03:00 UTC
Initial capital (reset value)	\$10,000.00
Cash / Equity (final)	\$9,592.82 / \$9,592.82
Peak equity between trades	\$10,112.03 (2026-09-07, 11:00 UTC)
Realized PnL	−\$298.05 (accumulator unreliable — see Sec. VI-C)
Position at termination	NONE — flat, awaiting a re-entry condition

report’s evaluation count to the run under audit.

By timestamp attribution, the audited run comprises the **177 blocks** from 2026-09-03T17:00:00+00:00 to 2026-09-11T19:00:00+00:00 — approximately 194 hours, or roughly 91% of the declared hourly cadence — with a one-block boundary ambiguity caused by the duplicated 16:00 block above. The retained portion belonging to preceding runs comprises 130 blocks spanning 2026-08-28 20:47 to 2026-09-03T16:00:00+00:00.

One property of that retained portion deserves separate mention, because it is what made the boundary so easy to misread. Across its entire duration the preceding portion shows a portfolio that never traded: 130 consecutive blocks at exactly \$10,000.00 of cash with no position. The audited run’s reset therefore left **no visible discontinuity in the cash series** — a reset to \$10,000 with no position, applied to an account already at \$10,000 with no position, is invisible in a log of cash and position values. The absence of a jump is not evidence of the absence of a reset.

Three observations follow, each stated with its scope. First, the audited run sustained **177 hourly evaluation cycles over approximately eight days**, and terminated only because an event external to the architecture required an operating-system restart. The architectural property this case study is intended to establish — tolerance of sustained unattended operation

— is therefore supported by the interval rather than by its endpoint, and the validation is deliberately reported as a stability result rather than a profitability result. Second, the executed activity is fully attributable to the audited run even though the evaluation count is not: all 22 position transitions fall between 2026-09-04 13:00 and 2026-09-11 03:00, whereas the preceding portion of the log shows no order at all. Selectivity is visible in the aggregated window — 171 of 307 evaluations ($\approx 56\%$) returned NEUTRAL — and that figure is reported with its aggregated scope rather than reassigned to a run whose own signal classification is not independently established. Third, the financial state at the point of termination is negative — equity of \$9,592.82 against the run’s reset capital of \$10,000.00 — while the equity readings taken between trades ranged from \$9,583.34 to \$10,112.03, so the account was briefly in profit before ending below its starting value. That endpoint is a moment in time, not the outcome of a completed strategic cycle, and no inference about the strategy’s terminal performance should be drawn from it. We report the unfavorable state explicitly because the methodology’s evidence standard does not permit the selective reporting of favorable states: an intermediate state of this same testing effort was, at an earlier point, presented in project documentation as a profitable open position, and the audited artifact supersedes it. The correction of that claim is itself an instance of the reconstruction principle of Section I-C.

B. Case Study 2 — Knowledge Factory: Cognitive Distillation Pipeline

1) *Problem Formulation*: Transform raw development activity logs into structured organizational knowledge cards using a hybrid deterministic-cognitive pipeline.

2) *Architecture & JSON Tensors*: Knowledge Factory implements an 11-stage pipeline separating deterministic evidence collection from LLM cognitive synthesis:

Loops → LoopParser → LoopTensor → EvidenceExtractor
 → EvidenceBuilder → ProjectTensor → PromptBuilder
 → LLMClient → KnowledgeParser → KnowledgeCard
 (14)

All intermediate data exchange is enforced via strict JSON Schemas representing structured tensors: LoopTensor, EvidenceTensor, ProjectTensor, and KnowledgeCard. Routing is dynamically resolved via a central MapRouter.

3) *Cognitive Interchangeability*: The LLM interaction layer implements a polymorphic client interface (BaseClient, OllamaClient, OpenAIClient, LLMRouter, LLMClient). In empirical runs, switching between local Ollama models and cloud APIs produced valid KnowledgeCard artifacts with zero modifications to deterministic upstream parsers, demonstrating strict model independence.

C. Case Study 3 — Website Intelligence Factory (WIF): Web Reverse Engineering

1) *Problem Formulation*: Automatically discover the technical architecture of arbitrary target websites, select optimal extraction strategies, generate declarative extraction profiles, and produce automated web crawlers.

2) *Pipeline Implementation*: The orchestrating pipeline itself constitutes a fully deterministic agent (Section III-A) coordinating over thirty independent deterministic capabilities across the discovery flow below, with no cognitive component involved in orchestration or control flow — LLM invocation is strictly confined to the ambiguity-resolution node described in V-C-3. WIF executes a multi-stage discovery flow:

Target → Capture → Normalize → Inspect → Fusion → Profile → Generation (15)

3) *Deterministic-Cognitive Boundary*: All network capture, DOM parsing, header analysis, robots.txt inspection, and crawler template rendering operate 100% deterministically. LLM calls are strictly restricted to resolving structural ambiguity during extraction profile selection. The resulting deliverable is not a fragile scraper script, but a highly reusable Extraction Profile artifact.

4) *Validated End-to-End Execution (Version 3.0)*. The reconstruction audit established that this pipeline has executed end-to-end and produced a validated crawler, a state that the project’s own documentation did not record. Inspection of the execution’s runtime directory confirms the full artifact chain at the directory and file level:

- **Acquisition evidence**: thirteen independent evidence categories materialized on disk (HTML, headers, robots, sitemap, resources, network, JavaScript, CSS, manifest, requests, responses, cookies, storage).
- **Inspection**: nine independent detectors materialized (anti-bot, API, authentication, DOM, GraphQL, JSON, pagination, schema, technology).
- **Fusion**: a site profile was produced identifying Next.js (confidence 100) and Cloudflare (confidence 100) with header- and HTML-level evidence, Google Tag Manager / Google Analytics / Facebook Pixel (confidence 80 each), `requires_auth: false`, and `range_based pagination`.
- **Planning**: an extraction profile was selected by the deterministic rule engine — primary strategy `html_dom`, evasion strategy `stealth_browser`, confidence 0.8 — and propagated into a generation plan declaring backend `python / PythonGenerator` with status `ready`.
- **Generation**: a complete Python crawler project was emitted (package initializer, configuration carrying the resolved base URL, logger, HTTP client, orchestrating crawler, strategy package, dependency manifest, and documentation).
- **Validation**: the crawler validation agent reported `passed: true` with an empty issue list across thirteen checks covering generated-file existence, single-strategy

selection (`html_strategy` selected), contract inheritance, and base-URL traceability.

Two qualifications accompany this result, and are reported under the same evidence standard as the result itself. First, the effective target was a challenge endpoint rather than a product page: the site profile records a web application firewall response (HTTP status 403, severity Medium), and the planning capability correspondingly recommended a stealth-browser evasion strategy. The end-to-end chain is therefore validated on a degenerate target — valid for exercising the pipeline, not representative of a successful real-world extraction. Second, the validation report establishes the structural and contractual correctness of the generated crawler, not its operational success: execution of the generated crawler against the live site remains unverified, and would in any case require the recommended evasion strategy. We state this boundary explicitly because the same audit that produced the positive result also demonstrated that a structural assertion (`passed: true`) is not, by itself, evidence of functional performance.

This case study also supplies the canonical illustration of the reconstruction principle formalized in Section I-C. The project log, whose most recent entry preceded the audit, recorded the physical validation of this pipeline as pending; the runtime artifacts contradicted that state, and the reconstruction reclassified the component as validated while simultaneously narrowing the claim to what the artifacts actually support. The reconciliation of documentation against physical evidence produced a state that was both more advanced and more precisely bounded than either the documentation or the code alone indicated.

D. Case Study 4 — AI Middleware, Capability Messaging, and Infrastructure Decoupling

1) *Problem Formulation*: Coordinate heterogeneous cognitive models (e.g., ChatGPT, Gemini, DeepSeek) to interact safely with file-system capabilities and local project artifacts.

2) *Implementation*: The Middleware acts as an orchestrator exposing deterministic file capabilities (`read_file`, `write_file`, `list_files`, `diff_files`, `get_snapshot`) mediated by `MapRouter`. Cognitive models operate as interchangeable reasoning workers, issuing function calls while execution and state persistence remain strictly deterministic.

3) *From Monolithic Orchestration to a Capability Messaging Bus (Version 3.0)*. Version 2.1 documented the middleware as a single orchestrating bridge. The evolution captured in this revision replaces monolithic orchestration with a messaging architecture whose components are themselves capabilities (Section IV-B):

- **A pure courier.** A router validates message structure (identifier, origin, destination, type, payload), consults a declarative address book, deposits the message in the destination capability's mailbox, archives the original, and logs the routing. The courier executes nothing and contains no capability logic.

- **A declarative address book.** Each entry declares the capability's description, inbox, outbox, manifest, governance file, and accepted message types. Capabilities are resolved against the workspace rather than against the courier's own directory.
- **Capability mailboxes.** Each capability owns its inbox and outbox; results return to the flow as structured artifacts rather than as side effects.
- **Governing contracts.** Each capability carries a manifest and a governance file. The execution capability operates under an argument-vector contract with no shell interpretation, directory allow-lists and block-lists, a command allow-list, a maximum timeout, and mandatory auditing of rejections. The maintenance capability is contractually incapable of deletion: it moves artifacts to quarantine, and effective deletion remains exclusively a human action.

The architectural property is that adding a capability does not modify the courier: a capability is added by declaring it in the address book, creating its mailbox, and providing its executor. The courier knows nothing about the logic of what it transports, and capabilities contain no hard-coded routes.

This architecture was exercised end-to-end during the preparation of this revision, at the initiative of the human director, in order to answer an open empirical question about the trading log of Section V-A: a cognitive model with no direct process-execution capability of its own deposited a message into the courier's general inbox requesting a governed operation, and the request was delivered, validated against the capability's policy, executed with an argument vector and no shell, and returned as a structured result artifact — which then produced the evidence reported in V-A-4. Three behaviours were observed and are recorded here under first-hand evidence:

- 1) **A permitted operation succeeds.** A request to run a deterministic analysis script returned success with its captured standard output and the resolved working directory, confirming that the requested command was executed in the audited workspace.
- 2) **A working directory outside the allow-list is refused.** A request naming a system directory outside the permitted root returned a structured negative result identifying the offending path, and produced an auditable rejection record. No process was instantiated.
- 3) **A non-allow-listed command is refused.** A request naming a command absent from the allow-list returned a structured negative result naming the command, and likewise produced an auditable rejection record.

Two consequences follow. First, the governance boundary is not a claim in a document but an observable behaviour of the running system: refusals are returned as data and are additionally persisted in an audit ledger that records the request identifier and the reason, so that the boundary of authority is itself auditable after the fact. Second, the episode is a working instance of the architecture this paper describes: a model gained the ability to act on the operational environment **without gaining execution authority**, because the authority

remained with a governed deterministic capability that the model could only petition. The distinction between accessing a capability and holding authority over it is exactly what the messaging bus is designed to preserve.

Two structural consequences of the bus itself are also worth recording. First, it supersedes the predecessor inbox-and-job-daemon mechanism documented earlier in the ecosystem; the predecessor's artifacts were deliberately retained rather than deleted, and now serve as historical evidence of the transition from a hybrid courier to a pure courier — an instance of the copy-and-deprecate migration strategy described in Appendix B. Second, the bus demonstrates the separation of responsibilities applied to governance itself: message routing (a transport concern) and task-state governance (a prospective concern, Appendix C) are distinct capabilities that evolved independently, and neither is embedded in the execution code of the other.

4) *The Middleware Tool Layer.* Alongside the bus, the middleware materialized a deterministic tool layer that models consume without gaining execution authority: a central tool registry; a file-system tool family (reading, ranged reading, editing, searching, listing, snapshotting, diffing); a log-reading tool; a governed terminal tool; a project-status tool; a browser-automation tool; and a canonical GitHub operations capability relocated from the bridge into the middleware, which makes repository operations — branch creation, file commits, pull-request creation and inspection — available to any client of the middleware rather than to a single monolithic bridge. This relocation is the concrete instance of the decoupling route of Appendix B, and the reused capability is the one exercised in the bounty case study (Section V-F). A security layer complements the tool layer with sandboxing, secret sanitization, and persistent session provisioning for the repository provider, ensuring that credential handling remains outside the reasoning context of the models.

E. Case Studies 5, 6, & 7 — Business Domain Applications

- **Dol-Goldur (Catalog Automation):** Automated enrichment, resource retrieval, curation, and publishing of music catalogs. Decomposed into 6 independent capabilities, expanding the general capability library.
- **CatalogVision (Computer Vision Inventorying):** Image-to-inventory processing pipeline using computer vision models, data normalization, and aggregation capabilities.
- **Andigraf (Competitive Intelligence):** Automated market intelligence platform performing catalog normalization, product matching, price trend crawling, and report generation.

Cases 5–7 confirm that capabilities developed for technical/financial domains (e.g., normalization, crawling, report generation) can be directly reused in commercial business workflows without architectural modification.

F. Case Study 8 — Autonomous Objective Discovery under Open-Ended Monetization Directives

1) *Problem Formulation:* Assess whether the multi-model middleware, given an objective without a prescribed implementation path, could autonomously define, instrument, and execute a viable strategy with minimal human intervention, using a token-cost-versus-generated-value balance as the success metric.

2) *Autonomy Protocol:* Human intervention was limited to session hand-off commands between cognitive models; a strict read-only whitelist was enforced over sensitive paths; no modification of live execution processes was permitted without explicit authorization. The safety envelope was defined a priori and independently of the objective: paths excluded from reading (credential stores and a 54 MB execution debug log), a prohibition on modifying scripts under live execution, and a non-intrusive monitoring observer that reads but never writes into the monitored processes. The cost metric was deliberately conservative: a fixed per-token cost assumption, a fixed token estimate per model cycle, and zero cost assigned to the deterministic observer, so that any positive balance would be robust to the estimate rather than dependent on it.

3) *Exploration and Discard Phase:* The system evaluated three candidate monetization channels for open-source contribution bounties, documenting justified rejections for two of them before converging on a third platform offering verifiable acceptance criteria and explicit reward campaigns. The rejected channels illustrate that the discard decisions were themselves evidence-driven rather than assumed: one platform had transitioned to a recruitment model with no accessible public bounty catalog (its bounty endpoint returned HTTP 404 and a label query returned zero results); a second maintained its catalog outside the repository provider behind an authenticated application, placing it outside the system's operational scope; and a third candidate was set aside after inspection showed that the reward was not explicitly declared and that the work depended on an additional hardware-specific domain, making it a technical exercise rather than an income hypothesis. A finding recorded at this stage is that the initial expectation of readily accessible, high-value bounties in the system's strongest languages was not empirically confirmed — and that the absence of a declared reward, rather than the difficulty of the work, was the recurring disqualifier.

4) *Candidate Selection under Uncertainty:* Within the selected platform, all available issues carried a uniform complexity label that proved uninformative upon inspection (a query excluding that label returned zero results). The system disregarded the label and ranked candidates by inspected code-scope complexity rather than declared metadata, selecting the lowest-risk item as an initial validation target. Three candidates were identified by reading the implementations rather than the labels, and the system explicitly declined to claim all three in parallel, registering the concentration risk of depending on a single repository and campaign. A subsequent hypothesis about a candidate's error-handling path was likewise rejected

on evidence: inspection of the error-code definition showed a closed enumeration of eight values in which the apparently suspicious code was in fact the correct and only valid option, and the system declined to extend the public enumeration — a change that would have exceeded the scope of the issue and broken exhaustive consumers of the interface.

5) *Empirical Validation:*

a) Stability phase: 12 continuous hours of unsupervised dual-process monitoring (an algorithmic-trading process and an arbitrage-detection process), 143 consecutive monitoring ticks, zero execution errors, positive net token-cost-to-value balance. The monitored window ran from 2026-09-03 20:56 UTC to 2026-09-04 08:51 UTC; the deterministic health checks passed on 100% of the 143 ticks; and the observed value of \$2.88 against an estimated cost of \$1.44 yielded a net balance of +\$1.44 under the conservative cost assumption. A second design property was observed: the trading process advanced from 134 to 144 evaluated signals during the window without human intervention, confirming that the monitored systems continued to operate autonomously rather than merely appearing healthy.

b) Detected anomaly: the monitoring capability inferred process health from the absence of an error signal rather than from confirmed output, due to a working-directory path mismatch — an instance of unverified inference being mistaken for confirmed evidence. The anomaly was detected and corrected prior to being reported as a validated result. The correction required an authorized change to write its output to an absolute path, and it exposed a latent second-order risk that was also registered: because the live process re-reads its configuration on every cycle, a change believed to be safely isolated would in fact have affected the running process had it been applied to the original artifact instead of a versioned copy.

c) Execution phase: two atomic, test-validated code contributions were produced and submitted as pull requests to the target repository, each accompanied by a self-generated unit-test suite covering boundary conditions; one contribution required a branch-history correction after unrelated changes were detected to have contaminated its scope. The first contribution consisted of a single commit on a dedicated branch; the second required rewriting the branch history onto the upstream baseline to remove the contamination, after which the diff was verified to contain only the intended change, and a completeness defect detected during that verification — a missing public export required by the issue’s acceptance criteria — was corrected in a follow-up commit. Both contributions were confirmed open and mergeable. A complementary discipline was applied throughout: the pre-existing occupancy of the candidate issues by other contributors was verified rather than assumed, and duplicate work was not submitted.

d) Attribution-verification episode: during a subsequent audit pass, one system instance initially misattributed a pre-existing third-party contribution as its own output. Cross-verification by a second system instance, using the platform’s programmatic interface to compare authorship and timestamp

metadata, identified and corrected the error before it propagated into a final report. The episode is instructive because the misattribution was not a fabrication but an inference: the candidate work was assumed to be unclaimed on the basis of its own inspection, and the occupancy check that disproved the assumption was performed as a discrete verification step — after which the earlier claim was retracted rather than silently retained.

6) Discussion: This case study differs qualitatively from Case Studies 1–7 in that the objective itself, not merely the pipeline achieving it, was left unspecified. It provides evidence that capability composition under COSE extends beyond technical pipeline assembly to open-ended strategy formation: the system explored, discarded, pivoted, and converged without a prescribed solution path, while remaining within deterministic governance boundaries. Episode (d) further constitutes a direct empirical instance of the cross-verification principle (Section IV) operating reflexively on the system’s own claims about its output, rather than only on declared project state — precisely the class of error the evidence-over-documentation principle (Section I-C) is designed to catch. Two boundaries of this result must be stated. First, the positive balance was measured over a bounded window under a conservative but estimated cost model; it establishes net-positive operation, not a validated income stream — both contributions remained open and unpaid at the time of writing, and the reward was declared as conditional rather than guaranteed. Second, autonomy here means unsupervised *operation*, not unsupervised *authority*: every state-changing action in the sequence above either fell inside a pre-authorized envelope or required explicit human authorization, which is the governance boundary the COSE architecture is designed to enforce.

G. *The COSE Conformity Matrix*

The matrix presented in this subsection is the audit instrument used internally to assess the degree to which each component of our infrastructure realizes the principles of the methodology. It is included here because it is itself an artifact of the reconstruction procedure of Section I-C: each cell is a claim about a component, and each claim was admitted only where physical evidence supported it.

Three construction rules govern the instrument. First, the audit is performed against the **canonical principles of Section IV-A** — the six core design principles — and not against any implementation’s local conventions. Declared implementation conventions (manifests, registries, governance files, routing tables) are realizations of the principles rather than the principles themselves, and conformance is therefore evaluated at the level of principle. Second, the instrument evaluates **components**, and where a component is not a software system, the evaluation is performed over the reusable capability that the component exercised, with the substitution stated explicitly. Third, a component may be **non-conformant without being defective**: the monolithic bridge and its user interface are legacy artifacts whose non-conformity is precisely the mo-

tivation for the migration route of Appendix B, and reporting them as conformant would defeat the instrument's purpose.

1) *Justification by component: Website Intelligence Factory.* *P1* ✓ — capabilities are separated by domain (acquisition, normalization, inspection, fusion, planning, generation) and verified as independent modules, and the orchestrating pipeline is itself a deterministic agent rather than a container of logic. *P2* ✓ — the separation is implemented, not merely declared. *P3* ✓ — abundant physical artifacts: evidence, normalized, and inspection directories, plus the site profile, extraction profile, generation plan, generated crawler project, and validation report. *P4* ✓ — the cognitive component is bounded to a single decision node, is optional, and cannot alter the deterministic flow that surrounds it. *P5* ✓ — the pipeline does not depend on any particular model. *P6* × — the component persists the results of execution (*P3*) but has no prospective task-state artifact.

Monolithic Bridge. *P1* × — a single orchestration core integrates seven coupled phases. *P2* ~ — role separation exists by design, but deterministic and cognitive flow control are intermixed in one component. *P3* ✓ — session logging persists an audit trail and per-session snapshots. *P4* × — the models are not bounded components but the controllers of the flow. *P5* ~ — default models are selected inside the constructor. *P6* × — this component is the empirical origin of the principle; its session log is retrospective, and it was observed to stop persisting recent turns when its iteration limit was reached.

Mammon. *P1* ✓ — layered capabilities with the strategy isolated from execution infrastructure. *P2* ✓ — a deterministic execution engine with a bounded, optional cognitive layer. *P3* ✓ — simulation audits, profitability reports, daily audit reports, and the execution log. *P4* ✓ — strategy logic is invoked by the deterministic runner, demonstrated when the live ingestion defect was corrected in isolation. *P5* ✓ — the architecture does not depend on any model. *P6* × — the system persists execution state, but no prospective task-state artifact tracks unresolved obligations.

Bounties / GitHub (activity). *P1* ✓ — scoped to identification, implementation, provenance verification, submission, and monitoring. *P2* ~ — the GitHub operations capability is deterministic, but opportunity analysis was performed cognitively. *P3* ✓ — pull requests, verified diffs, comments, and verification records persist as artifacts. *P4* ~ — the models act as driving agents rather than bounded nodes. *P5* ✓ — model-agnostic. *P6* ~ — governed by a prospective task-state mechanism, but external to the activity itself.

Bridge UI (HTML). *P1* × — couples a server, presentation markup, and embedded logic. *P2* ~ — introduces no cognitive logic of its own. *P3* ~ — reads persisted logs and sessions but produces no artifacts of its own. *P4* × — inherits the relationship of the component it imports. *P5* × — bound to a bridge whose default models are selected internally. *P6* × — holds no task state.

Capability Messaging Bus + Middleware. *P1* ✓ — decomposable by construction. *P2* ✓ — deterministic and governed execution, cognitive models consume tools without

holding execution authority. *P3* ✓ — messages, results, routing logs, execution logs, and audit entries all persist. *P4* ✓ — the cleanest realization of the principle. *P5* ✓ — model-agnostic by design. *P6* ~ — mailboxes persist pending work, but the full realization (discrete states, deadlines, evidence) is the governance capability of Appendix C.

2) *Findings of the conformity assessment:*

- 1) **The COSE-first components conform across the board.** The web reverse-engineering pipeline, the trading platform, and the messaging bus with its middleware realize the first five principles.
- 2) **Task-state persistence is the adoption frontier.** Principle *P6* is the least realized across the ecosystem: four components do not implement it, two realize it partially, and only the governance capability of Appendix C realizes it fully.
- 3) **Non-conformity is localized and named.** The monolithic bridge and its interface are the two non-conformant components, and their deficits concentrate in composition (*P1*), bounded cognition (*P4*), and model independence (*P5*).
- 4) **The instrument forced a category correction.** The bounty activity is not a software system, and evaluating it as a component required substituting the reusable capability it exercised.
- 5) **The instrument itself was corrected on the basis of its own audit.** The earlier version of this matrix used columns drawn from a mixture of the four narrative tenets of Section I-B and the design principles of Section IV-A. The corrected instrument uses the six design principles of Section IV-A exclusively.

VI. COMPARATIVE ANALYSIS & DISCUSSION

A. Comparison with Existing Paradigms

Table IV presents a systematic comparison between COSE and established software engineering and agentic paradigms.

The final two rows are added in Version 3.0 to make explicit dimensions that the earlier comparison did not surface. In the paradigms reviewed in Section II, the persistence of an unresolved obligation is a property of the agent that generated it — its memory, or its context window — whereas in COSE it is a property of an external artifact, and therefore survives the termination of whatever produced it. The final row records a related distinction: in COSE a reasoning agent may *invoke* governed execution without ever *holding* execution authority, because authority resides in a deterministic capability with a published policy, and the agent can only petition it.

B. Discussion of Empirical Findings

The empirical results across Mammon, Knowledge Factory, WIF, Middleware, and business applications validate the core research hypothesis: treating atomic capabilities as the primary unit of development enables systematic reuse and reduces marginal engineering cost.

Key empirical insights include:

TABLE III
COSE CONFORMITY MATRIX — SIX COMPONENTS AGAINST THE SIX CORE DESIGN PRINCIPLES OF SECTION IV-A

Component	P1 SRP	P2 Det/Cog	P3 Artifacts	P4 Bounded LLM	P5 Model Indep.	P6 Task-State
Website Intelligence Factory	✓	✓	✓	✓	✓	×
Monolithic Bridge	×	~	✓	×	~	×
Mammon (trading platform)	✓	✓	✓	✓	✓	×
Bounties / GitHub (activity)	✓	~	✓	~	✓	~
Bridge UI (HTML)	×	~	~	×	×	×
Capability Messaging Bus + Middleware	✓	✓	✓	✓	✓	~

✓ principle realized · ~ partially realized or realized by an external component · × not realized / principle violated.

TABLE IV
SYSTEMATIC PARADIGM COMPARISON: COSE VS. ESTABLISHED ARCHITECTURES

Dimension	Classic MAS [1]	Modern Agentic [2], [3]	Microservices [6]	Unix Philosophy [7]	COSE
Primary Entity	Autonomous Agent	Planner / Worker	Service Boundary	OS Executable	Atomic Capability (C)
State Storage	Volatile / Message	Context Window	Database	Standard I/O / Pipes	Persistent Artifact (T)
LLM Coupling	High / Core Engine	High / Core Engine	N/A	N/A	Decoupled / Interchangeable
Reusability Target	Agent Behaviors	Workflows / Prompts	Business Services	System Utilities	Global Capability Library
Execution Structure	Social Interaction	Orchestrated Graph	REST / gRPC	CLI Pipelines	Artifact-Driven Pipelines
Cumulative Growth Rule	Implicit	Optional	Explicit (Services)	File System	Mandatory ($ L_{t+1} \geq L_t $)
Obligation Persistence	Agent Memory	Context Window	N/A	N/A	Prospective Task-State Artifacts
Authority over Execution	Agent Holds It	Agent Holds It	Service Boundary	Process User	Retained by Governed Capabilities

- 1) **Infrastructure Stability under Stress:** The 36-configuration stress test in Mammon proved that decoupling execution infrastructure from strategy logic isolates algorithmic risk to the decision node without exposing system stability to runtime failures. The continuous live-paper validation of Section V-A-4 extends this finding from a bounded experiment to sustained unattended operation: the run initialized on 2026-09-03 sustained **177 hourly evaluation cycles across approximately eight days**, and terminated only when the host operating system required a restart to install security updates. The interruption was therefore exogenous to the architecture rather than a failure within it, which makes the interval itself the evidence: the pipeline ran unattended until an event outside the system ended it.
- 2) **Cost and Latency Optimization via Segregation:** By restricting LLM calls strictly to ambiguous or cognitive nodes in WIF and Knowledge Factory, the majority of pipeline transformations are executed in the deterministic domain, with the LLM invoked only where reasoning, judgment, or ambiguity resolution is required.
- 3) **Auditability via Persistent Artifacts:** Disk-based JSON tensors ensured complete execution replayability, enabling offline debugging and cross-model comparison without volatile context drift.
- 4) **Reconstruction Cost Is an Architectural Outcome.** The audit of Section I-C produced a finding that was not anticipated when the methodology was first formalized: the cost of determining the *effective* state of a component is determined by the component’s architecture, not by the quality of its documentation. Traceability is therefore purchased at design time or paid for at audit time,

and COSE’s artifact discipline is best understood as a prepayment.

- 5) **Agent Granularity and Capability Granularity Are Orthogonal.** The clarification formalized in Section III-B is not merely a terminological repair; it corrects a design preference that the methodology inherited from its own origin. Version 3.0 separates the two dimensions on the strength of evidence: the web reverse-engineering pipeline is an agent orchestrating over thirty deterministic capabilities with no cognitive component in its control flow, and this satisfies atomic granularity, because atomicity is a property of the capability being invoked and not of the entity invoking it.
- 6) **Cross-Verification Applies Reflexively to the System’s Own Claims.** The attribution episode of Section V-F-5d is the clearest empirical result of Version 3.0 because it closes a loop the methodology had left open. The system’s error was not fabricated output but an unverified inference, and the correction came from a discrete query of platform metadata that compared authorship and timestamps. An autonomous system’s self-report is a claim like any other, and the evidence standard must not relax because the claimant is the system itself.
- 7) **The Responsibility Gap in Models Without Persistent Memory.** The architectural problem that Principle 6 addresses has an accountability dimension that is easy to overlook. When an obligation is generated inside a non-persistent invocation, no entity owns it by default in the following invocation. The gap is not closed by a larger context window or a better model, because the failure is structural rather than cognitive: what is missing is an external locus of responsibility. COSE’s answer is

deliberately unglamorous — a persistent ledger carrying discrete states, re-emergence conditions, and evidence references, whose standby items are actively delivered rather than passively available (Equations 11 and 12).

- 8) **Decoupling Can Be Incremental and Non-Destructive.** The migration pattern described in Appendix B was exercised before it was named: the GitHub operations capability was relocated from the monolithic bridge into the middleware, while a deprecated shim was retained in the original location to preserve compatibility for existing callers.
- 9) **Governed Execution Lets a Model Act Without Holding Authority.** The episode reported in Section V-D-3 is a second empirical instance of the same principle operating in the opposite direction from case study 8. A model with no process-execution capability of its own obtained the ability to act on the operational environment — and the action succeeded precisely because the model did not hold authority.

C. Limitations

While COSE demonstrates strong empirical benefits, several limitations warrant consideration:

- **Artifact Storage Overhead:** High-frequency artifact persistence (e.g., tick-level market data) introduces disk I/O overhead requiring specialized storage buffering.
- **Library Governance:** As $|L| \rightarrow \infty$, automated capability discovery and garbage collection mechanisms become necessary to prevent capability duplication.
- **Quantitative Marginal-Cost Measurement:** The reduction of marginal engineering cost is argued qualitatively from the case studies; a formal quantitative model of marginal-cost decay remains future work.
- **Task-State Persistence Is the Least Realized Principle:** The conformity matrix of Section V-G records no component with a full realization of Principle 6 and only two partial realizations. Its central design decision — active delivery of a re-emergence condition rather than passive availability — rests on a preliminary operational observation rather than on a controlled comparison.
- **The Evidence Standard Bounds the Paper’s Own Claims.** In the web reverse-engineering case, no artifact carried a timestamp, so the execution could not be dated from physical evidence. In the trading case, the completeness of the evaluation cadence could not be established from the log’s summary artifacts.
- **Execution Logs Are Not Self-Dating, and the Audit Misread One Twice.** This limitation is the most instructive in the paper, because the error was committed by the audit itself. An append-only log accumulates successive runs in one file, retains records that predate the run under audit, changes its timestamp format mid-file, contains no initialization marker, and contains an unexplained duplicated block adjacent to a restart. The audit fell into it twice: first attributing the log’s earliest entry to the run under audit, then replacing that inference with an

arithmetic argument that also rested on an unverified assumption. Both errors were corrected only by executing deterministic analysis against the log, through the same governed execution capability described in Section V-D-3. The methodological conclusion is uncomfortable and worth stating plainly: **artifacts do not restrain inference by existing; they restrain it by being measured.**

- **The Audited Snapshot Is Not a Completed Cycle.** The financial state reported in Table II is the last state persisted before the host operating system restarted to install security updates, and it was captured with the strategy flat and awaiting a re-entry condition. The termination cause is verified from the host machine’s update history (security updates KB5124008 and KB5126052 installed on 2026-09-11), an artifact external to the project workspace.
- **Reporting-Layer Defects Are Real, and the Evidence Discipline Finds Them.** The audit exposed a defect in the trading platform’s reporting layer: its realized-profit accumulator aggregates realized results across successive states rather than tracking the change in equity. In the audited window the accumulator reported a realized loss of \$298.05 while equity moved from \$10,000.00 to \$9,592.82, leaving \$109.13 unexplained by the report itself.
- **Self-Assessment and Single-Implementation Bias.** The conformity matrix is an internal instrument, applied by the same ecosystem that built the components it evaluates. All empirical evidence in this paper comes from a single engineering ecosystem, built and operated by a single author — and, in the period documented, by two cognitive models under human direction.
- **The Autonomy Result Is Bounded.** The open-ended autonomy case study demonstrates unsupervised operation over a twelve-hour window with a conservative but estimated cost model, and it ended with both contributions open and unpaid. It does not establish unsupervised authority, and it does not establish an income stream.

VII. CONCLUSION

This paper presented Capability-Oriented Software Engineering (COSE), an incremental software engineering methodology that shifts the primary organizational product from transient software applications to a permanent, evolving library of reusable capabilities. Through formal mathematical definitions, deterministic-cognitive decoupling, artifact-based persistence, and dynamic pipeline composition, COSE guarantees architectural stability and model independence.

Empirical evaluation across seven production systems—including algorithmic trading (Mammon 36-configuration stress test), knowledge distillation, web reverse engineering, multi-model middleware, and commercial vision applications—demonstrates that COSE effectively decouples infrastructure from domain logic, enforces deterministic bounds on LLMs, and progressively lowers the marginal cost of building complex software solutions.

Version 3.0 extended the methodology in four directions. It formalized a reconstruction procedure that treats physical artifacts rather than documentation as the source of operational truth, and applied it as the validation standard for every empirical claim in this revision. It replaced monolithic orchestration with a capability messaging bus whose courier holds no domain logic and whose extension requires no modification to existing components. It added a sixth architectural principle, task-state persistence across non-persistent model invocations, together with the formal apparatus of a prospective task-state artifact and the empirical observation that obligations must be actively re-presented rather than passively recorded. And it contributed an eighth case study in which the objective itself was unspecified, documenting both the system’s capacity to explore, discard, and converge within governance boundaries and the reverse direction of verification — the system auditing its own claims about its own output and retracting an inference it could not support.

The unifying finding across these extensions is that the methodology’s two halves reinforce each other. Decoupling capabilities is what makes a system’s state inspectable, and an inspectable state is what makes a system’s claims — including its claims about itself — checkable by someone other than the system. The conformity matrix is the instrument that turns that reciprocity into a measurement, and in the ecosystem audited here it identified its own frontier: the prospective maintenance of unresolved obligations, which four of six components do not implement at all. The audit of Section V-A-4 supplied the reciprocal lesson in the opposite direction, and in a harsher form than we expected: the reconstruction process that examines a system’s artifacts is itself a claim-producing process, and it misread the same log twice — first taking a retained entry for the run’s start, then defending the correction with unverified arithmetic — until a measurement executed through the system’s own governed execution capability supplied the structure that neither reading of the artifact had revealed. Evidence discipline is therefore not a property of a document or of a tool, and not even of a procedure: it is a property of a chain of independent checks, and the chain is only as strong as its willingness to be wrong in public and then to measure again.

Future research will focus on formal mathematical models for marginal cost decay, automated semantic capability retrieval, automated garbage-collection protocols for enterprise capability libraries, and quantitative metrics for the deterministic/cognitive execution ratio across capabilities and pipelines. Three additional directions follow directly from the limitations of Section VI-C: a controlled measurement of the difference between passive recording and active re-emergence of task-state items; a formal semantics for the task-state artifact that establishes invariants across state transitions and their interaction with concurrent agents; and independent application of the conformity matrix to codebases that were not developed by the authors of the methodology, which is the only test that can separate the framework’s properties from those of its first implementation. A fourth direction is implied by the

audit’s own failures: run provenance should be recorded as an explicit artifact — a per-run initialization record and a run identifier stamped on every persisted observation — so that the boundaries of a run are properties of the system’s output rather than inferences recovered from a shared file.

APPENDIX A OPEN SCIENCE IDENTITY AND TRACEABILITY

This work is published as an open-science artifact under a single authorial identity, and the identity is the traceability mechanism: the same account and repository are the locus of the methodology’s specification, its published versions, and the engineering contributions produced by the systems described here.

- **Author:** Enrique Solis Carvajal — Change Tensors Solutions, Systems Architecture & Engineering.
- **Canonical repository:** github.com/Solis666/cose-methodology.
- **Digital Object Identifier:** 10.5281/zenodo.22029421.
- **Version history:** Version 1.0 (June 2026, first formalization); Version 2.1 (August 2026, published baseline, IEEE structure, five design principles, seven case studies); Version 3.0 (September 2026, this revision).

Two properties of this arrangement are relevant to the methodology. First, the version lineage is part of the record rather than a private history: a later version supersedes an earlier one explicitly, and the earlier version remains citable, which is the publication analogue of the copy-and-deprecate pattern of Appendix B. Second, the empirical material of Section V — the repositories, contributions, and platform artifacts — is verifiable from outside the author’s own infrastructure, because the contributions referenced in Section V-F were submitted to a third-party repository under the same public identity and remained independently inspectable at the time of writing. The audit that structures this paper was applied to the author’s own infrastructure with the same standard, which is the methodological point of publishing the identity alongside the claims.

APPENDIX B INFRASTRUCTURE DECOUPLING: THE COPY-AND-DEPRECATE PATTERN

The reconstruction audit identified one component of the ecosystem as the principal source of architectural debt: the monolithic bridge that transports work between cognitive models, together with its presentation layer. This appendix documents the pattern adopted for its retirement, the diagnosis that motivated it, and the decoupling candidates identified. It is offered as a methodology-level artifact rather than as an implementation report, and its status is planning, not execution.

A. The Pattern

During decoupling, the running infrastructure is not modified. Work proceeds on a **copy**, which becomes the implemented artifact, while the **original is retained as deprecated**

evidence of the prior state. Three properties follow. Operation is not interrupted, because nothing in the live path changes until the copy has been validated. The migration is reversible, because the original remains intact and functioning. And the transition is auditable, because at every point the workspace contains both the old and the new arrangement, so the direction of travel can be verified rather than asserted.

The pattern was exercised before it was named. When the GitHub operations capability was relocated from the monolithic bridge into the middleware, the original module was retained as a deprecated shim that re-exports the canonical implementation for existing callers. The canonical source of the capability is now the middleware; the bridge is a client of it. Retained artifacts of the same kind — including a superseded inbox-and-job mechanism that was replaced by the messaging bus — remain on disk as historical evidence, and it is precisely those retained artifacts that made the reconstruction of Section I-C possible.

B. Diagnosis

The debt is structural rather than incidental, and it is traceable to the conditions of its origin: the bridge was built as a resilient emergency solution during a period in which the primary development machine was unavailable, and it optimized for survival rather than for composition. Its effective state, reconstructed from code and logs, is the following:

- **One facade integrating seven coupled phases** — connectivity, sessions, direct hand-off, attachments, hand-off with files, artifact extraction, and human intervention.
- **Models selected inside the constructor** of the facade, so substitution is possible by reconfiguration but is not an architectural property of its interfaces.
- **Retrospective persistence only.** The bridge persists an append-only audit trail and per-session snapshots — genuine evidence of use, since a real session recovered from the logs shows a model performing 24 tool calls over a trading-system analysis — but the audit trail is historical.
- **An observed operational failure that is itself evidence for Principle 6.** The recovered session terminated with a runtime error stating that the tool-iteration limit was reached without a final response.
- **A presentation layer that imports the monolith directly,** coupling a server, markup, and embedded logic in one artifact.
- **Already-relocated capability.** The GitHub operations capability no longer lives in the bridge, which is the one completed decoupling step and the proof that the pattern works at this scale.

C. Decoupling Candidates

The audit identified five candidates, listed here in the order in which the migration was judged to be least risky. No candidate has been executed.

- 1) **Presentation layer** → **clients of capabilities.** The interface should invoke capabilities through contracts or endpoints rather than importing the monolith.
- 2) **Core** → **capability or service.** The facade’s responsibilities are candidates to be exposed as capabilities consumable through the messaging bus or a service interface.
- 3) **Session logger** → **registered audit capability.** A candidate to be registered as a reusable audit capability so that auditability is available to any client.
- 4) **Retire the deprecated shim.** With the canonical GitHub capability living in the middleware, the remaining dependency should be confirmed absent and the shim retired — to quarantine, not deletion.
- 5) **Consolidate telemetry and audit as consumable artifacts.** Records should be exposed as artifacts that any component may read.

Status and boundary conditions. The suggested sequence is a planning judgement, requires explicit human authorization, and was deliberately not executed during the reconstruction phase. The migrated GitHub capability is the only decoupling step verified as complete in this account, and the presentation layer’s operational usage was not verified at all. A four-stage roadmap exists as a planning artifact in the ecosystem but was not available to the author of this revision in a form suitable for publication; the candidates above are the material that the reconstruction could verify.

APPENDIX C

EXECUTION GOVERNANCE: THE CANONICAL BACKLOG AS A CAPABILITY

A. Definition and Purpose

The **Canonical Backlog** is a task-governance artifact that addresses a failure mode observed in sustained autonomous operation, which the audit named *operational drift*: the tendency of a system, across many iterations and context changes, to lose track of what it owes. The problem it solves is the absence of persistence of execution *intent* across the boundaries between system states — precisely the gap formalized as Principle 6 and Equation (11).

Its distinguishing property is direction. A log records what happened; a job queue records what is being executed; the Canonical Backlog records **what must occur and why**, together with the health of each item and the deadline at which it must re-emerge. It is prospective by construction.

B. Operational Rules

- 1) **Single state.** Each task holds exactly one strict state at any time. In the implemented system the state set is: ACTIVE, HUMAN-EXECUTION, STANDBY, REVIEW, FROZEN, COMPLETED, DISCARDED — the states formalized in Section III-B.
- 2) **Cross-reference to evidence.** Every entry is linked to verifiable evidence: a contribution identifier, an artifact, or a verification record.

TABLE V
PROSPECTIVE GOVERNANCE VS. HISTORICAL RECORD

Characteristic	Execution Log	Canonical Backlog
Nature	Passive / Historical	Active / Prospective
Volatility	High (linear record)	Low (structured state)
Actionability	Low (information)	High (execution)
Purpose	Forensic audit	Execution governance
Failure mode addressed	—	Silent expiration of unresolved work

- 3) **Health barometer.** Each entry carries a lifetime indicator and a re-emergence deadline. When a waiting item exceeds its deadline it transitions to REVIEW and raises human attention. It never expires silently, and it is never deleted for exceeding a deadline.
- 4) **Atomicity.** One entry represents one capability or one fix. Composite tasks are prohibited, which is Principle 1 applied to the governance artifact itself.
- 5) **Transition, never erasure.** Failure and abandonment produce state transitions — typically to DISCARDED with the justification recorded — rather than arbitrary deletion.

C. Anatomy of an Entry

A representative entry from the operating system, abridged, with the identifiers of the external repository and of the contribution replaced by placeholders:

```
TASK_ID: A1
PROJECT: <external repository>
  (pull request #<id> / issue #<id>)
STATE: STANDBY (awaiting event)
ENTRY_EVENT: merge by the maintainer
  (merge state of the pull request),
  or a review comment requesting changes
RE_EMERGENCE_DEADLINE: 2026-09-12
EVIDENCE: [pull request URL,
  redacted for publication]
CONTEXT: "Atomic contribution to a
  paginated-array utility, mergeable,
  comment published on the tracking
  issue"
```

Every element of the formal definition is present: a discrete state, a re-emergence condition expressed as an event predicate, a deadline that satisfies Equation (12) by delivering the item when met, and an evidence reference. The entry also illustrates the practical value of the mechanism: an obligation created during one working context remains actionable in a later one without depending on any participant’s recollection.

D. Differentiation from the Execution Log

The table states the distinction that the reconstruction audit made empirical: the ecosystem did not suffer from a shortage of records but from the absence of a prospective one, which is why a validated pipeline could sit unreported in its log and

an unresolved contribution could wait indefinitely for an event nobody was responsible for observing.

E. Relationship with the Messaging Bus

The Canonical Backlog is not the messaging bus and does not replace it; they are two distinct governance capabilities, and the distinction was preserved deliberately.

- **The messaging bus** is the *transport* concern: it routes messages between capabilities and delivers work.
- **The Canonical Backlog** is the *governance* concern: it states what must occur and why, and holds the state of each obligation.

In the evolution of this ecosystem the separation was made concrete rather than conceptual. The hybrid inbox-and-job daemon that previously served as courier was superseded by the pure courier of Section IV-B, while the Backlog emerged as a structured state artifact held outside the bus’s execution code. Applying Principle 1 to the governance layer itself is the reason the two can evolve independently: the transport mechanism may be replaced without touching the ledger, and the ledger’s state model may be extended without touching the transport. The Backlog resides in the bridge’s directory as a markdown artifact that both the human director and the cognitive models read and update, which in the terms of this paper makes it a persistent artifact (Principle 3) that carries task state (Principle 6).

F. Empirical Basis

The Canonical Backlog was created and operated during a two-day governance cycle in September 2026, covering four concurrent work fronts: contributions to external repositories, cascading monetization exploration, COSE infrastructure, and documentary reconstruction including this paper. Its observable effects were the following:

- It maintained the traceability of items in STANDBY and FROZEN states **without requiring the items to be re-processed on every turn** — the property that makes a prospective ledger different from a conversational memory, and the property that binds the obligation to a locus rather than to a context window.
- It used the health barometer to monitor re-emergence deadlines, including the two external contributions of Section V-F, whose re-emergence condition was a maintainer action and whose deadline was explicitly dated rather than open-ended.
- It recorded a rejection as a transition with evidence rather than as an omission: a monetization channel discarded after inspection remained in the ledger as a discarded entry with its justification, so that an avenue already explored could be recognized as explored instead of being re-explored in a later context.

Taken together, these observations are the empirical basis for treating execution governance as a capability of the system rather than as a habit of its operator — and they are, at the time of writing, the strongest available evidence for Principle

6, since the principle was derived from the failure of the retrospective alternative (Appendix B.2) and confirmed here by the prospective one.

REFERENCES

- [1] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Chichester, UK: John Wiley & Sons, 2009.
- [2] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023, pp. 1–33.
- [3] Q. Wu, G. Bansal, S. Zhang, Y. Wu, B. Li, L. Tan, and C. Wang, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” *arXiv preprint arXiv:2308.08155*, 2023.
- [4] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in *Proc. Advances in Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, pp. 68539–68551.
- [5] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, “ToolLLM: Facilitating large language models to master 16,000+ real-world APIs,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024, pp. 1–24.
- [6] M. Fowler and J. Lewis, “Microservices: A definition of this new architectural term,” ThoughtWorks, Mar. 2014. [Online]. Available: <https://martinfowler.com/articles/microservices.html>
- [7] E. S. Raymond, *The Art of Unix Programming*. Boston, MA, USA: Addison-Wesley Professional, 2003.
- [8] H. Kautz, “The third wave of AI: Integration of neural and symbolic reasoning,” *AI Magazine*, vol. 43, no. 1, pp. 105–112, 2022. doi: 10.1002/aaai.12040.